

# Bit-Noise Immunity via Double-Redundant Guard

The applied fix treats malformed or extreme input as **bit-noise** and eliminates it before it reaches the memory allocation layer.

## 1. The Core Problem: Silent Truncation (Noise)

On a 32-bit architecture, any product exceeding  $2^{32} - 1$  undergoes silent wraparound:

$$\text{Signal} = \prod (ne_i \cdot \text{type\_size}), \quad \text{Noise} = \text{Signal} \bmod 2^{32}$$

A genuine 4.1 GiB tensor is then perceived as  $\approx 100$  MiB, causing the allocator to return a buffer that is orders of magnitude too small, resulting in heap corruption or a misleading OOM error.

## 2. Double-Redundant Defence

### 2.1 Physical Layer (64-bit Buffer)

Declaring the accumulator as `uint64_t` provides a computational headroom of  $2^{32}$  times the addressable space of a 32-bit host. The exact product is preserved throughout the multiplication chain:

$$\text{data\_size}_{64} \in [0, 2^{64} - 1]$$

### 2.2 Logical Layer (Pre-condition Discriminator)

Before each multiplication, a boundary check is applied:

$$\text{GGML\_ASSERT} \left( ne[i] = 0; \vee; \text{data\_size} \leq \frac{\text{SIZE\_MAX}}{(\text{size\_t}), ne[i]} \right)$$

This  $O(1)$  check guarantees that the 64-bit accumulator never contains a value that cannot be safely cast back to the 32-bit `size_t`. It acts as a **noise-injection filter**: if a dimension value is large enough to cause an eventual overflow, it is detected **before** the operation corrupts the state.

## 3. Empirical Validation (Stress-Test)

A synthetic stress-test simulated a 32-bit environment with  $\text{MAX\_ALLOC\_SIZE} = 2^{32} - 1$ .

Three scenarios were evaluated:

| Test                     | Dimensions                         | Element size | Total bytes        | Expected behaviour |
|--------------------------|------------------------------------|--------------|--------------------|--------------------|
| 1 (safe)                 | $4 \times 4096 \times 16$          | 64 B         | 16 MiB             | Pass               |
| 2 (unexpected overflow)  | $2000 \times 2000 \times 500$      | 16 B         | 29.8 GiB           | Intercepted        |
| 3 (exact limit overflow) | row_size = 1 GiB,<br>$ne = [2, 2]$ | 1 GiB        | exactly $2^{32}$ B | Intercepted        |

### Test 2 output (abridged):

The interception triggers a `GGML_ASSERT`, ensuring the process terminates **before** any `malloc` call is attempted with truncated values. The abort is a deliberate safety measure, not an uncontrolled crash. This confirms that the guard is **hypersensitive** and prevents any silent corruption regardless of whether the input is malicious or an honest miscalculation.

In every case, the 64-bit accumulator correctly held the full value, and the pre-condition check stopped execution before any memory corruption could occur.

---

## 4. Efficiency Gain over Previous Clamping

---

Earlier approaches required a post-multiplication clamp and a ternary selection on the final assignment. The new logic moves the verification into the loop where the dimension values are already being processed, re-using the existing loop infrastructure. The final assignment simplifies to a safe, direct cast:

```
obj_alloc_size = (size_t)data_size;
```